SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1513

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**
Question Count: 5

Code of Conduct

I SIDDHI SRIVASTAVA P(192521454) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: Siddhi

Assessment Tasks

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
   The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
   They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
   The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
   Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented.
   Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured.
   Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

SET 2:

1.A retail company collects terabytes of customer transaction data every day from online and offline stores. The management wants to analyze purchasing patterns in near real time to improve product recommendations and inventory management.

- Design a cloud-based big data architecture using distributed storage and processing frameworks.
- Explain how batch processing and stream processing can be integrated.
- Discuss how data lakes and data warehouses can be used together.
- Recommend techniques to ensure scalability and cost efficiency.

2.A healthcare organization is migrating patient records and medical imaging data to the cloud. The solution must comply with strict privacy regulations while allowing authorized doctors to access records securely from multiple locations.

- Design a secure cloud architecture that protects sensitive healthcare data.
- Explain how encryption, identity and access management (IAM), and key management should be implemented.
- Discuss auditing, logging, and compliance monitoring strategies.
- Recommend methods to prevent unauthorized access and data breaches.

3. An online ticket booking platform experiences sudden spikes in traffic whenever popular events are announced. The company wants to reduce infrastructure management while ensuring fast response times.

- Design a serverless cloud architecture for the application.
- Explain how event-driven computing can improve scalability.
- Discuss the role of managed databases, API gateways, and message queues.
- Evaluate the advantages and limitations of serverless computing for this workload.

4. A multinational company gathers data from sales, finance, manufacturing, and customer support systems. The organization requires a centralized analytics platform with high-quality and consistent data.

- Design a cloud-based ETL/ELT pipeline for integrating data from multiple sources.
- Explain how data quality, metadata management, and governance should be maintained.
- Discuss the role of workflow orchestration and scheduling tools.
- Recommend strategies for optimizing query performance and storage costs.

5. A logistics company wants to predict delivery delays using historical shipment data, weather information, and live GPS tracking. The analytics platform must support continuous model training and real-time predictions.

- Design a cloud-based machine learning and analytics architecture.
- Explain how big data technologies can support model training and inference.
- Discuss the use of cloud-native AI/ML services for model deployment and monitoring.
- Recommend strategies for handling large-scale data ingestion, feature engineering, and model updates.

SET 3 :

1. A manufacturing company plans to migrate its legacy enterprise applications and databases from an on-premises data center to the cloud. The organization wants to minimize downtime while ensuring application performance and security.

- Design a cloud migration strategy suitable for the organization.
- Explain how different migration approaches (Rehost, Refactor, Replatform, Repurchase, Retire, Retain) can be applied.
- Discuss methods to validate application performance after migration.
- Recommend strategies to reduce migration risks and downtime.

2. A smart city project deploys thousands of IoT sensors to monitor traffic, air quality, energy consumption, and public safety. The collected data must be processed and analyzed in real time for decision-making.

- Design a cloud-based architecture for IoT data collection and analytics.
- Explain how message brokers, stream processing, and cloud storage are integrated.
- Discuss methods for handling high-volume, high-velocity sensor data.
- Recommend techniques to ensure scalability, reliability, and security.

3. A global e-commerce company wants executives to monitor sales performance, customer behavior, and regional trends through interactive dashboards. The solution must support real-time analytics and historical reporting.

- Design a cloud-based business intelligence architecture.
- Explain how data visualization tools integrate with cloud data warehouses.
- Discuss techniques for optimizing dashboard performance with large datasets.
- Recommend security measures to protect business reports and sensitive data.

4. An online gaming company experiences fluctuating workloads throughout the day. During peak hours, users report high latency and reduced application performance.

- Design a cloud monitoring and performance optimization solution.

- Explain how cloud monitoring tools, metrics, logs, and tracing help identify performance bottlenecks.
- Discuss strategies for resource optimization and capacity planning.
- Recommend methods to improve application availability while controlling operational costs.

5. A financial institution stores petabytes of customer transaction data for regulatory compliance and business analytics. The organization must retain historical records while managing storage costs efficiently.

- Design a cloud-based data lifecycle management strategy.
- Explain how data classification, retention policies, and archival storage should be implemented.
- Discuss the importance of governance, auditing, and regulatory compliance in big data environments.
- Recommend techniques for balancing data accessibility, security, and cost optimization.

SET 1

Question 1: Video Streaming Platform Migration

Scenario: A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering.

1. Cloud Architecture Using Scalability and Caching Techniques

The platform is split into a delivery path (what viewers touch) and an ingestion path (how content is prepared). Every component in the delivery path is stateless or cache-backed so that capacity can be added horizontally without redesign.

- **Edge delivery:** A Content Delivery Network (CDN) caches video segments (HLS/DASH chunks) at edge locations close to viewers. Popular titles are served almost entirely from the edge, which removes most traffic from the origin and reduces buffering.
- **Adaptive bitrate streaming:** Each video is transcoded into several bitrates and resolutions. The player switches quality automatically with network speed, preventing stalls.
- **Application tier:** Stateless API and streaming servers run in an Auto Scaling Group across multiple Availability Zones behind an Application Load Balancer.
- **Caching layers:** An in-memory cache (Redis/Memcached) stores sessions, catalogue metadata and recommendations so that the database is not hit on every request. Browser and CDN caching handle static assets.
- **Storage:** Master and transcoded videos reside in highly durable object storage, which acts as the origin for the CDN. A managed database holds users, subscriptions and viewing history.
- **Ingestion pipeline:** Uploaded content goes to a queue, and a fleet of transcoding workers (often Spot instances) processes it and writes the output to object storage.

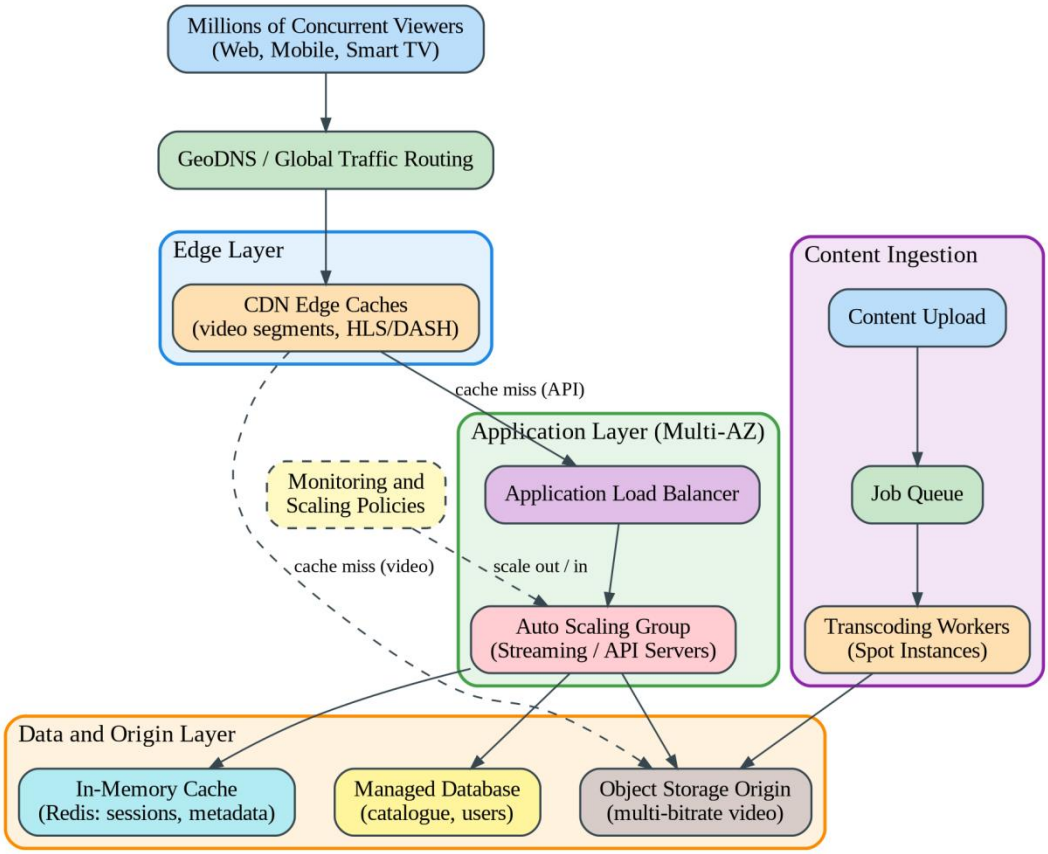


Figure 1: Scalable Video Streaming Architecture with CDN, Load Balancing and Auto-Scaling

2. Load Balancing and Auto-Scaling for Smooth Streaming

Load balancing distributes requests across healthy servers. GeoDNS or a global load balancer sends each viewer to the nearest region, and the Application Load Balancer spreads requests across instances in several Availability Zones using least-connections or round-robin. Health checks remove failed servers automatically, and connection draining lets in-progress streams finish before an instance is retired.

Auto-scaling adjusts capacity to demand. Target-tracking policies keep average CPU, network throughput or concurrent streams per instance near a target value. Scheduled scaling prepares extra capacity before known events such as a live match or a major series release, while predictive scaling uses historical patterns. When load rises, new instances are launched from a pre-baked image within minutes; when it falls, extra instances are terminated.

Together, the two mechanisms ensure that no single server becomes a bottleneck, that failures are invisible to viewers, and that start-up delay and buffering stay low even during sudden audience surges.

3. Cost Optimization Strategies for High-Demand Workloads

Strategy	How It Is Applied	Benefit
Right-sizing	Choose instance types matched to the workload (compute-optimised for streaming, GPU or media-optimised for transcoding).	Avoids paying for unused capacity.

Strategy	How It Is Applied	Benefit
Reserved / Savings Plans	Commit to the steady baseline load for one to three years.	Large discount on predictable usage.
Spot instances	Run fault-tolerant transcoding and batch jobs on spare capacity.	Up to about 70 to 90 percent saving.
Aggressive CDN caching	Long cache lifetimes for video segments and a high cache hit ratio.	Reduces origin load and data transfer cost.
Storage tiering	Move old or rarely watched titles to infrequent-access or archive storage via lifecycle rules.	Lower storage cost with no manual work.
Efficient encoding	Use modern codecs (H.265/AV1) and per-title encoding.	Less bandwidth per stream.
Monitoring and budgets	Cost dashboards, tagging, budget alerts and regular reviews.	Prevents unexpected spending.

Question 2: Containers Across Multiple Cloud Providers

Scenario: A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments.

1. Solution Using Container Orchestration and a Multi-Cloud Strategy

The inconsistency arises because each provider has different tooling, networking and deployment methods. The solution is to standardise on **Kubernetes** as a common control layer and manage everything declaratively.

- **Containerisation:** Every service is packaged as an immutable container image, so it runs identically on any provider.
- **Orchestration:** A Kubernetes cluster is created in each cloud (managed services such as EKS, AKS and GKE reduce operational effort). Multi-cluster tools such as Rancher, Anthos or Azure Arc give a single management view.
- **GitOps deployment:** Desired state (Helm charts, manifests) is stored in Git, and a controller such as Argo CD or Flux applies it to every cluster. This guarantees the same configuration everywhere and makes rollback a simple Git revert.
- **CI/CD and registry:** A pipeline builds, tests and scans images, signs them and pushes them to a replicated registry that all clusters pull from.
- **Infrastructure as Code:** Terraform provisions networks, clusters and IAM identically across providers.
- **Cross-cloud traffic management:** A global load balancer or DNS service directs users to the closest or healthiest cluster.
- **Central security and observability:** Vault for secrets, OPA/Gatekeeper for policy, and Prometheus, Grafana and ELK for unified monitoring.

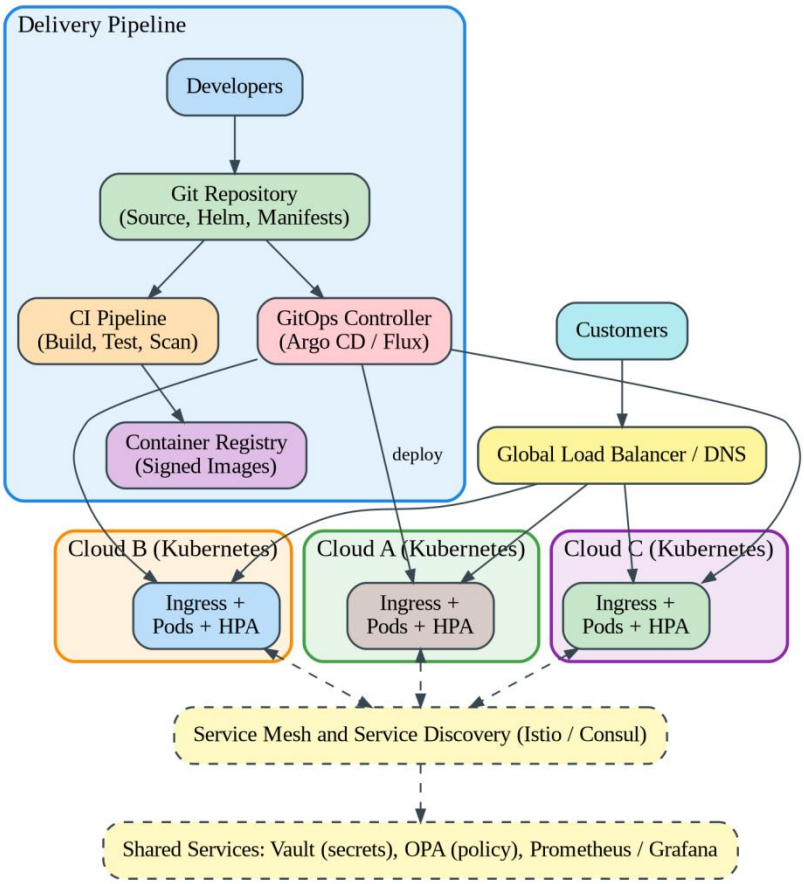


Figure 2: Multi-Cloud Kubernetes Deployment with GitOps and Service Mesh

2. Service Discovery and Scaling

Service discovery: Inside a cluster, Kubernetes Services and CoreDNS give every group of pods a stable DNS name, even though pod IP addresses change. Across clusters and clouds, a service mesh (Istio, Linkerd) or Consul provides cross-cluster discovery, mutual TLS between services, retries and traffic shifting. Ingress controllers and API gateways expose services externally.

Scaling: The Horizontal Pod Autoscaler adds or removes pods based on CPU, memory or custom metrics such as requests per second. The Cluster Autoscaler adds worker nodes when pods cannot be scheduled and removes idle ones. KEDA can scale from queue length or events. Resource requests and limits, pod disruption budgets and readiness probes keep scaling safe for payment-critical services.

3. Benefits and Risks of the Multi-Cloud Deployment Model

Benefits	**Risks**
Avoids vendor lock-in and improves bargaining power.	Higher operational complexity and need for skilled staff.
Resilience: a provider outage does not stop the business.	Inconsistent security models and a larger attack surface.

Benefits	Risks
Use of the best service or region from each provider.	Data egress charges and latency between clouds.
Helps meet data-residency and regulatory requirements.	Compliance and audit effort multiplied across providers.
Cost optimisation by choosing cheaper capacity.	Tendency to use only the lowest common denominator of services.

For a fintech, the benefits outweigh the risks only if standardisation (Kubernetes, Terraform, GitOps) and strong central governance are in place.

Question 3: Backup and Disaster Recovery for an E-Learning Platform

Scenario: An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy.

1. Solution Using Cloud-Native Backup, Replication and Versioning

Accidental deletion is a logical error, so replication alone is not enough because a deletion would be replicated too. The design therefore combines versioning, independent backups and replication.

- **Versioning:** Object storage has versioning enabled, so deleting or overwriting a file only creates a new version or delete marker and the earlier version can be restored. MFA Delete and restrictive IAM policies prevent unauthorised permanent deletion.
- **Cloud-native backup:** A managed backup service takes scheduled snapshots of databases and volumes, with point-in-time recovery enabled for the database. Backups follow a 3-2-1 approach: three copies, two storage types, one off-site.
- **Immutable backup vault:** Backups are stored in a separate account with write-once-read-many (WORM) locking, protecting them from accidental or malicious deletion.
- **Replication:** Cross-region replication copies objects, and the database has an asynchronous cross-region replica, protecting against a regional failure.
- **Lifecycle policies:** Older backups move to cheaper archive tiers and expire according to the retention policy.
- **Governance:** Least-privilege IAM, a recycle-bin or soft-delete period, and alerts on bulk deletion events.

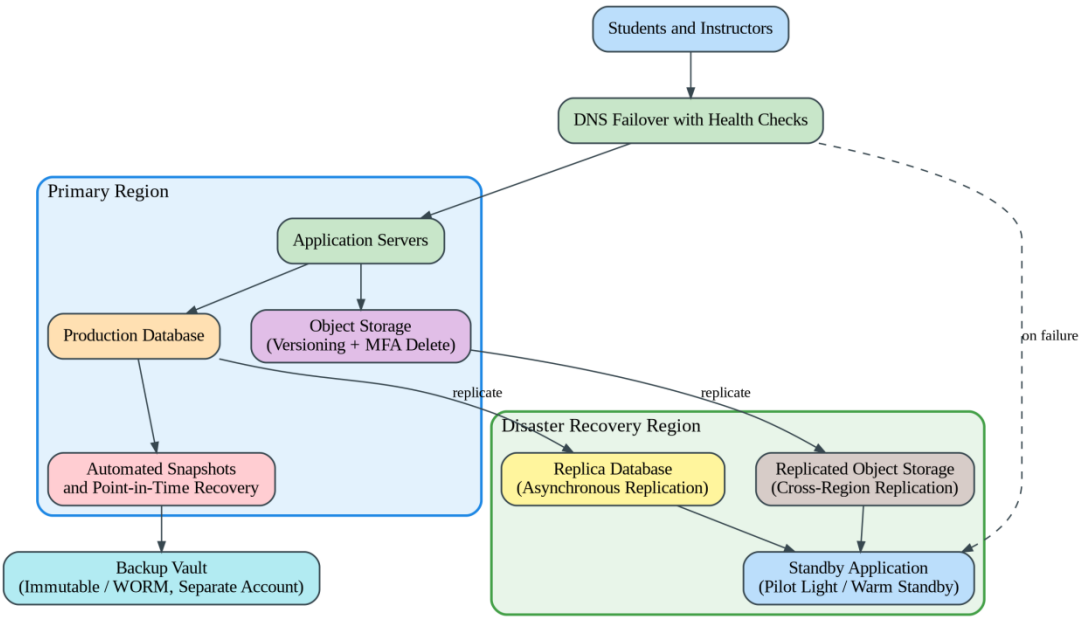


Figure 3: Backup and Disaster Recovery Architecture with Versioning, Immutable Vault and Cross-Region Replication

2. How RTO and RPO Are Achieved

Recovery Point Objective (RPO) is the maximum acceptable amount of data loss, measured in time. **Recovery Time Objective (RTO)** is the maximum acceptable downtime. Each is met by choosing mechanisms that match the target.

Objective	Mechanism	Typical Outcome
RPO (data loss)	Continuous replication, frequent snapshots, point-in-time recovery, object versioning.	Seconds to minutes of data loss.
RTO (downtime)	Pre-provisioned standby environment, infrastructure as code, automated failover through DNS health checks, tested runbooks.	Minutes to a few hours depending on standby level.

For example, if the platform sets an RPO of 5 minutes and an RTO of 1 hour, it uses continuous database replication plus point-in-time recovery for the RPO, and a warm standby application in the second region with automated failover for the RTO. Stricter targets cost more, so critical systems (grades, payments) receive tighter targets than low-priority content.

3. Steps to Ensure Business Continuity During Failures

- Perform a business impact analysis and classify systems by criticality, then assign RTO and RPO values.
- Document a disaster recovery plan with roles, escalation contacts and step-by-step runbooks.
- Monitor health continuously and raise alerts automatically when a failure is detected.
- Declare the incident and fail over: DNS is switched to the standby region and the replica database is promoted.

- Restore deleted or corrupted data from versions or backups to the required point in time.
- Communicate status to students, instructors and management through a status page.
- Verify the restored services with health and data-integrity checks before reopening traffic.
- Fail back to the primary region when it is stable, then review the incident and update the plan.
- Test regularly through restore drills and game days, because untested backups are not reliable.

Question 4: Hybrid Cloud for a Government Agency

Scenario: A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements.

1. Hybrid Architecture with Secure Communication

Data is classified first. Sensitive and compliance-bound workloads stay in the on-premises data centre, while non-sensitive and elastic workloads such as public portals, analytics and development environments run in the public cloud.

- **Connectivity:** A site-to-site IPsec VPN provides an encrypted tunnel, and for production a dedicated private link (such as Direct Connect or ExpressRoute) provides predictable latency and bandwidth. Traffic never traverses the public internet unprotected.
- **Network segmentation:** Separate virtual networks and subnets, security groups, network ACLs and firewalls restrict traffic to only the required ports and services. Sensitive systems use private endpoints with no public IP.
- **Perimeter protection:** A web application firewall, DDoS protection and intrusion prevention guard cloud-facing applications.
- **Data placement:** Sensitive records remain on-premises. If cloud applications need them, only tokenised or masked data is exposed through secured APIs.
- **Central visibility:** Logs from both environments flow to a single SIEM for monitoring and audit.

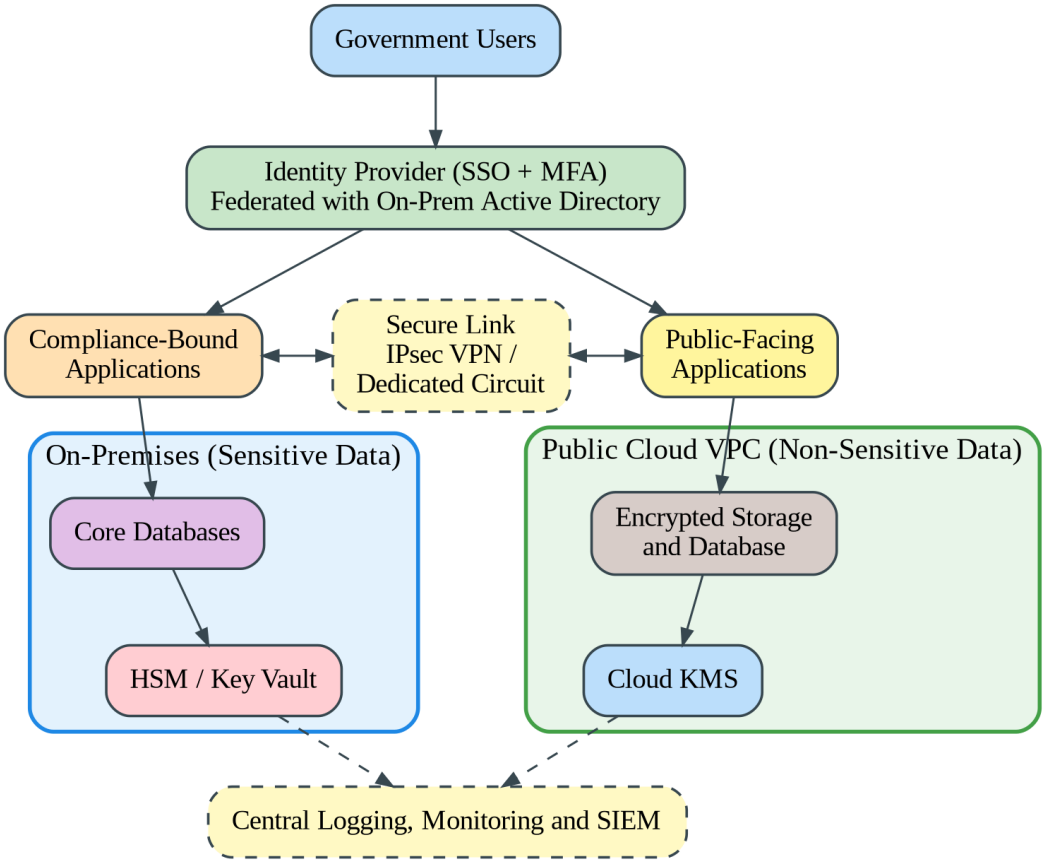


Figure 4: Hybrid Cloud Architecture with Secure Connectivity, Federated Identity and Encryption

2. Identity Management and Encryption

Identity management: The on-premises Active Directory is the source of truth and is federated with the cloud identity service using SAML or OpenID Connect, giving single sign-on with one identity across both environments. Multi-factor authentication is mandatory, role-based access control enforces least privilege, privileged access management and just-in-time access control administrators, and joiner-mover-leaver processes remove access promptly. Service accounts use managed identities rather than hard-coded credentials.

Encryption: Data in transit is protected with TLS 1.2 or higher between users and applications, and IPsec across the hybrid link. Data at rest is encrypted with AES-256 in databases, disks and object storage. Keys are held in a Hardware Security Module on-premises or in a cloud KMS using customer-managed keys, with rotation, separation of duties and audit logging of key use. For the highest sensitivity, the agency keeps keys on-premises (bring-your-own-key or hold-your-own-key).

3. Challenges in Managing Hybrid Cloud Environments

Challenge	Explanation and Mitigation
Integration complexity	Different platforms and APIs. Use common tools such as Kubernetes, Terraform and a hybrid management platform.
Consistent security and compliance	Policies can drift between environments. Apply policy as code and continuous compliance scanning.

Challenge	Explanation and Mitigation
Network latency and bandwidth	Chatty applications suffer across the link. Use dedicated links, caching and careful workload placement.
Unified monitoring	Separate tools hide problems. Aggregate metrics and logs into a single observability platform.
Skills and operating model	Teams need both on-premises and cloud skills. Provide training and clear shared responsibility.
Data governance and cost	Data sprawl and egress charges. Maintain data classification, tagging and cost controls.

Question 5: High Availability and Fault Tolerance for a SaaS Provider

Scenario: A SaaS provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if one region goes down.

1. Multi-Region Deployment Strategy

The application is deployed in at least two geographically separate regions, and within each region across multiple Availability Zones, so that no single failure takes the service down.

- **Active-active:** Both regions serve traffic, giving the best availability and near-zero RTO, but it needs data conflict handling and costs more. A global traffic manager uses latency or geolocation routing.
- **Active-passive (warm standby):** One region serves traffic while the other runs a scaled-down copy that scales up on failover. It is cheaper but has a slightly longer recovery time.
- **Stateless application tier:** Sessions are kept in a shared cache or tokens so any region can serve any user.
- **Infrastructure as code and identical pipelines:** Both regions are built from the same templates and deployed from the same CI/CD pipeline, avoiding configuration drift.
- **Dependencies:** Container registries, secrets, DNS and identity services are also replicated or made multi-region, otherwise they become hidden single points of failure.

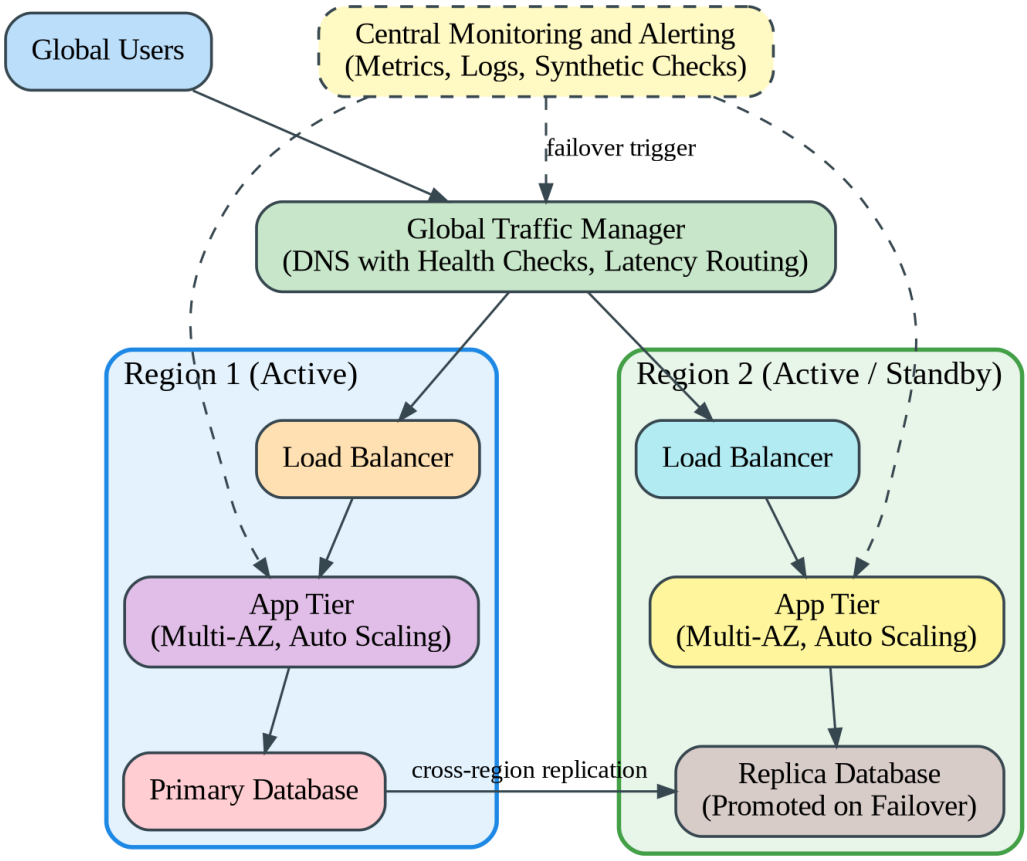


Figure 5: Multi-Region Active-Active Deployment with Replication and Failover

2. Data Replication and Failover Configuration

Data replication: For relational databases, a primary in one region with asynchronous cross-region read replicas gives low latency and an RPO of seconds, while synchronous replication within the region protects against zone failure. Globally distributed databases (such as DynamoDB Global Tables, Cosmos DB or Spanner) allow multi-region writes with defined conflict resolution. Object storage uses cross-region replication, and caches are rebuilt or replicated. Backups are kept as an additional safeguard against corruption.

Failover mechanisms: Health checks probe endpoints in each region. When the checks fail repeatedly, DNS or the global load balancer stops routing to the failed region within seconds to minutes, using a short DNS time-to-live. The replica database is promoted to primary (automatically or through a controlled runbook), the standby application scales out, and queues and background jobs resume in the surviving region. Failback is done only after the original region is verified healthy and data is resynchronised. Regular failover drills validate the configuration.

3. Monitoring and Alerting for Proactive Issue Detection

- **Metrics:** Track the golden signals (latency, traffic, errors, saturation), replication lag, queue depth and resource usage per region.
- **Logs and traces:** Centralise logs and use distributed tracing to find the failing component quickly.

- **Synthetic monitoring:** External probes simulate user journeys from several locations and detect outages before customers report them.
- **Alerting:** Alerts are based on service-level objectives and error-budget burn rate, with severity levels, escalation policies and on-call tools (PagerDuty, Opsgenie). Alerts should be actionable to avoid fatigue.
- **Anomaly detection and capacity forecasting:** Machine-learning-based alarms and trend analysis flag problems before they become incidents.
- **Automation:** Self-healing actions such as restarting unhealthy instances or triggering failover, plus dashboards and a public status page.
- **Post-incident review:** Blameless reviews feed improvements back into the design.

SET 2

Question 1: Big Data Analytics for a Retail Company

Scenario: A retail company collects terabytes of customer transaction data every day from online and offline stores. Management wants to analyze purchasing patterns in near real time to improve product recommendations and inventory management.

1. Cloud-Based Big Data Architecture

The architecture uses distributed storage and distributed processing so that it scales horizontally with data volume.

- **Ingestion:** Point-of-sale systems, the e-commerce site, mobile apps and inventory systems publish events to a distributed messaging service (Apache Kafka, Amazon Kinesis or Google Pub/Sub). Batch extracts from legacy systems arrive through managed data transfer tools.
- **Distributed storage:** A data lake on object storage (S3, ADLS, GCS) or HDFS stores raw data cheaply and durably in open formats such as Parquet.
- **Distributed processing:** Apache Spark (EMR, Databricks, Dataproc) processes large datasets in parallel, and Flink or Spark Structured Streaming handles events as they arrive.
- **Serving:** A low-latency store (Redis, Cassandra, DynamoDB) serves recommendations and live stock levels, and a data warehouse serves analytical queries.
- **Consumption:** Recommendation engine, inventory replenishment alerts and BI dashboards.

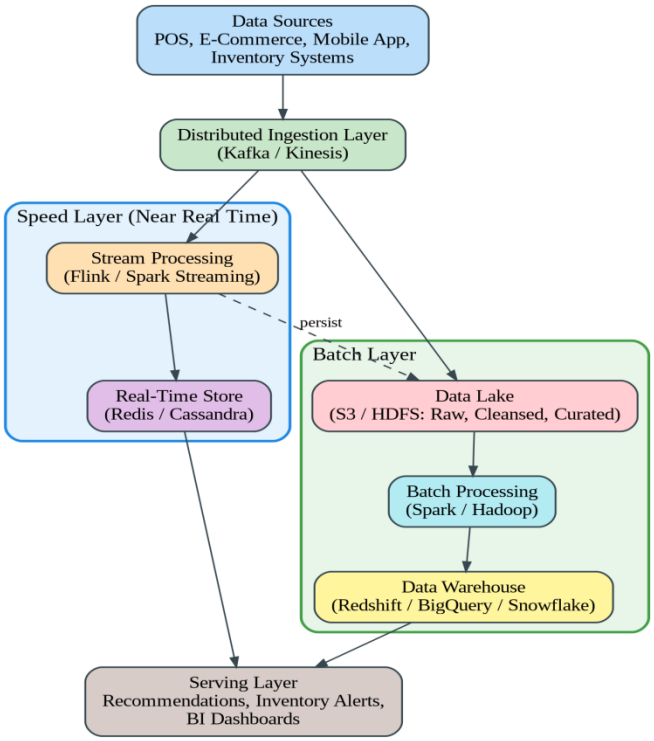


Figure 6: Retail Big Data Architecture Integrating Batch and Stream Processing

2. Integrating Batch and Stream Processing

The two modes are combined using the **Lambda architecture** (or the simpler **Kappa architecture** where one streaming engine handles both).

- The **speed layer** processes each transaction within seconds to update running aggregates, trigger low-stock alerts and refresh recommendation features.
- The **batch layer** periodically reprocesses the complete history in the data lake to compute accurate aggregates, train recommendation models and correct late or duplicate events.
- The **serving layer** merges both views, so users see fast but approximate results from streaming and accurate, complete results from batch.
- Modern lakehouse formats (Delta Lake, Apache Iceberg, Hudi) allow the same table to be written by streaming jobs and read by batch jobs, reducing duplicated logic.

3. Using Data Lakes and Data Warehouses Together

Aspect	**Data Lake**	**Data Warehouse**
Data type	Raw structured, semi-structured and unstructured data (logs, JSON, images).	Cleaned, structured, modelled data.
Schema	Schema on read.	Schema on write.
Users	Data engineers and data scientists.	Business analysts and managers.
Cost	Very low storage cost.	Higher cost, optimised for fast SQL.
Typical use	Machine learning, exploration, long-term retention.	Dashboards, reports, KPIs.

In practice, all data lands in the lake first. Spark jobs clean and enrich it into curated zones, and the business-ready subset is loaded into the warehouse (Redshift, BigQuery, Snowflake) for fast reporting. Many organisations use a lakehouse pattern or query the lake directly from the warehouse (external tables), so both are used together instead of duplicating data.

4. Techniques for Scalability and Cost Efficiency

- Use autoscaling clusters and serverless engines so that compute scales with load, and shut down idle clusters.
- Separate storage from compute so each scales and is billed independently.
- Use columnar formats (Parquet, ORC) with compression, partitioning by date or store and clustering to reduce scanned data.
- Apply storage lifecycle policies that move old data to cheaper tiers.
- Use Spot or pre-emptible instances for batch jobs and reserved capacity for steady workloads.
- Pre-aggregate and cache frequent queries with materialized views.
- Monitor cost with tagging and budgets, and drop unused data and pipelines.

Question 2: Secure Cloud Architecture for Healthcare Data

Scenario: A healthcare organization is migrating patient records and medical imaging data to the cloud. The solution must comply with strict privacy regulations while allowing authorized doctors to access records securely from multiple locations.

1. Secure Cloud Architecture for Sensitive Healthcare Data

The design follows defence in depth and zero-trust principles, and uses a cloud region and services that are eligible for the relevant regulations (HIPAA, GDPR or national health data laws).

- **Network isolation:** Databases and storage sit in private subnets with no public IP. Access is through private endpoints, security groups and network ACLs.
- **Secure entry point:** Doctors connect through a web application firewall and API gateway over TLS, and optionally a VPN or zero-trust access proxy for remote locations.
- **Application tier:** Hardened, patched servers or containers run the EHR and image viewer (PACS/DICOM), authenticated against the central identity provider.
- **Data tier:** An encrypted managed database for patient records, and encrypted object storage for large medical images, with lifecycle tiers for old studies.
- **Resilience:** Multi-AZ deployment, encrypted backups and a disaster recovery region in the same legal jurisdiction.
- **Segregation:** Separate accounts or projects for production, development and security tooling, and de-identified data for analytics or research.

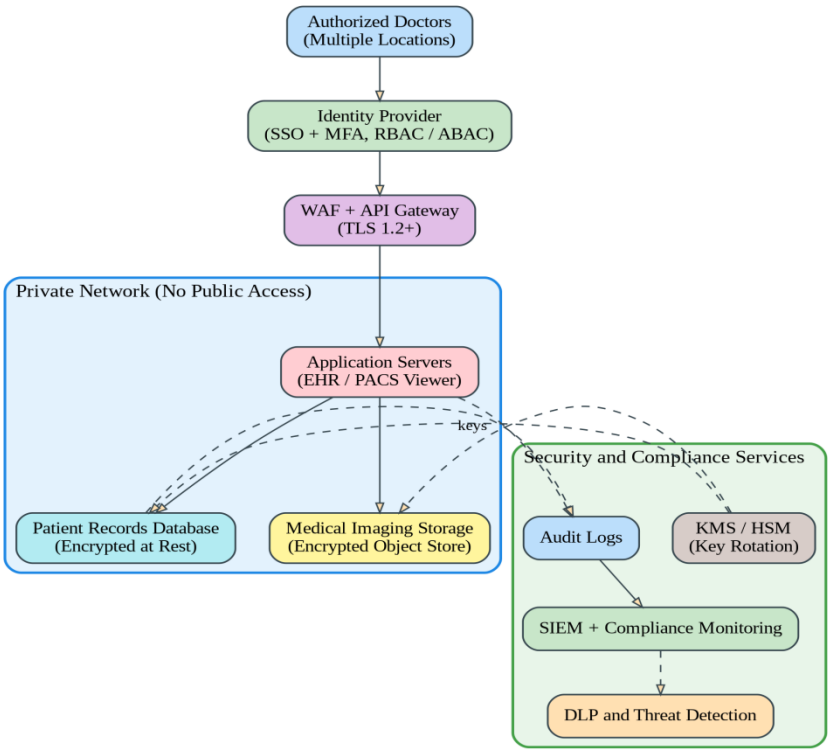


Figure 7: Secure Healthcare Cloud Architecture with IAM, Encryption and Audit Logging

2. Encryption, IAM and Key Management

Encryption: Data in transit uses TLS 1.2 or higher, and internal service traffic uses mutual TLS. Data at rest uses AES-256 for databases, volumes, object storage and backups. Sensitive fields such as national ID can additionally use field-level encryption or tokenisation.

Identity and Access Management: Doctors authenticate with single sign-on and multi-factor authentication. Role-based and attribute-based access control give each user the minimum access required (for example, a doctor sees only the patients under their care). A break-glass procedure allows emergency access with extra logging and review. Privileged administrative access is time-limited and approved, and service accounts use short-lived credentials.

Key management: Keys are created and stored in a managed Key Management Service or HSM, preferably as customer-managed keys. Policies enforce automatic rotation, separation of duties between key administrators and data users, and revocation. Every key use is logged. For stricter requirements, keys can be held in an external HSM controlled by the organisation.

3. Auditing, Logging and Compliance Monitoring

- **Audit trails:** Record every access to patient data (who, what, when, from where) and every administrative or API action in tamper-proof, immutable log storage with retention that meets the regulation.
- **Centralised logging and SIEM:** Collect logs from applications, databases, network and identity services, and apply correlation rules to detect suspicious behaviour such as unusual record access or large downloads.
- **Compliance monitoring:** Use cloud security posture management and configuration rules to detect public storage, unencrypted resources or open ports and remediate automatically.
- **Reporting and reviews:** Scheduled access reviews, automated compliance reports, vulnerability scanning, penetration tests and external audits.
- **Data governance:** Data classification, retention and a breach notification procedure that meets regulatory deadlines.

4. Methods to Prevent Unauthorized Access and Data Breaches

Layer	Preventive Measures
Identity	MFA, least privilege, conditional access by device and location, quick revocation of leavers.
Network	Private networks, firewalls and WAF, DDoS protection, micro-segmentation, no public storage.
Data	Encryption everywhere, DLP, masking and tokenisation, backups with immutability.
Application	Secure coding, input validation, dependency and container scanning, regular patching.
Detection and response	SIEM alerts, anomaly detection, an incident response plan with regular drills.

Layer	Preventive Measures
People	Security awareness and phishing training, strict vendor and third-party controls.

Question 3: Serverless Architecture for Ticket Booking

Scenario: An online ticket booking platform experiences sudden spikes in traffic whenever popular events are announced. The company wants to reduce infrastructure management while ensuring fast response times.

1. Serverless Cloud Architecture

- **Frontend:** The static website or single-page application is stored in object storage and delivered through a CDN, so page loads are fast and never touch servers.
- **Authentication:** A managed identity service (such as Amazon Cognito or Firebase Auth) handles sign-up, login and tokens.
- **API layer:** An API gateway receives requests, validates tokens, applies throttling and caching, and invokes functions.
- **Compute:** Function-as-a-Service (AWS Lambda, Azure Functions, Cloud Functions) implements search, seat reservation, booking, payment and notification logic.
- **Messaging:** A queue and event bus decouple booking from payment and notifications.
- **Data:** A managed NoSQL or serverless relational database stores events, seats and bookings, and an in-memory cache holds seat availability.
- **Notifications:** Email and SMS services send tickets and confirmations.

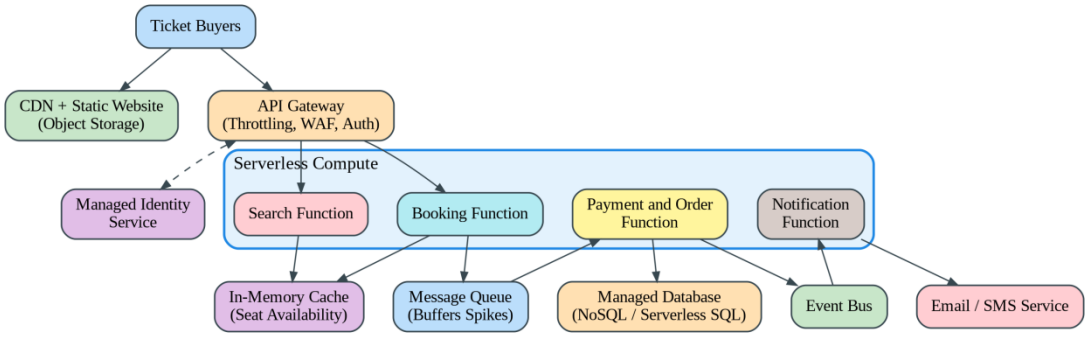


Figure 8: Serverless Ticket Booking Architecture

2. How Event-Driven Computing Improves Scalability

In an event-driven design, components react to events (an API call, a message in a queue, a database change, a payment confirmation) instead of running continuously. Each event can trigger its own function instance, so when thousands of users book at once the platform simply runs thousands of function instances in parallel, and when traffic stops the compute drops to zero and costs nothing.

- **Automatic, fine-grained scaling** with no capacity planning or servers to patch.

- **Loose coupling:** the booking function only publishes an event, and payment, ticket generation and email each subscribe independently, so a slow component does not block the others.
- **Resilience:** failed events are retried and sent to dead-letter queues rather than lost.
- **Pay-per-use** billing aligned with actual requests.

3. Role of Managed Databases, API Gateways and Message Queues

Component	Role in the Ticket Booking Platform
Managed database	Provides automatic scaling, replication, backups and patching. Conditional writes or transactions prevent two users from booking the same seat. Caching reduces read load during spikes.
API gateway	Single entry point that handles authentication, rate limiting and throttling, request validation, caching, versioning and protection against abuse and bots.
Message queue	Absorbs sudden bursts by buffering requests so that backend functions process them at a controlled rate. Supports retries, ordering where needed (FIFO queues), and a virtual waiting room for very popular events.

4. Advantages and Limitations of Serverless for This Workload

Advantages	Limitations
Instant, automatic scaling for unpredictable spikes.	Cold-start latency on first invocation, which can be reduced with provisioned concurrency.
No server management, so the team focuses on features.	Execution time, memory and concurrency limits; very high concurrency may need quota increases.
Pay only for what is used, which is very cheap between events.	Cost can exceed containers for constant, very high traffic.
Built-in high availability across zones.	Vendor lock-in due to proprietary services.
Faster development and deployment.	Harder debugging and testing across distributed functions; database connection limits must be managed.

Overall, serverless is a strong fit for this spiky, event-driven workload, provided that cold starts, seat-locking logic and database limits are designed for.

Question 4: Cloud ETL/ELT Pipeline for a Multinational Company

Scenario: A multinational company gathers data from sales, finance, manufacturing, and customer support systems. The organization requires a centralized analytics platform with high-quality and consistent data.

1. Cloud-Based ETL/ELT Pipeline Design

- **Extract:** Managed connectors and change data capture (Fivetran, AWS Glue, Azure Data Factory, Debezium) pull data from CRM, ERP, manufacturing execution systems and the support ticketing tool, in batch or incrementally.

- **Load (raw zone):** Data is first landed unchanged in a cloud data lake, preserving the original for audit and reprocessing.
- **Transform:** In the **ELT** approach, transformations run inside the scalable warehouse or lakehouse engine using SQL or Spark (for example dbt). Data moves from the raw zone to a cleansed zone (deduplicated, validated, standardised currency, units and time zones) and then to a curated zone (business models such as customer, product and order).
- **Serve:** Curated data is loaded into a cloud data warehouse, with star-schema data marts for sales, finance, manufacturing and support.
- **Consume:** BI dashboards, self-service analytics and machine learning.
- **ETL versus ELT:** Traditional ETL transforms before loading and suits strict pre-load masking; ELT loads first and uses the cloud's elastic power for transformation, which is usually faster and more flexible.

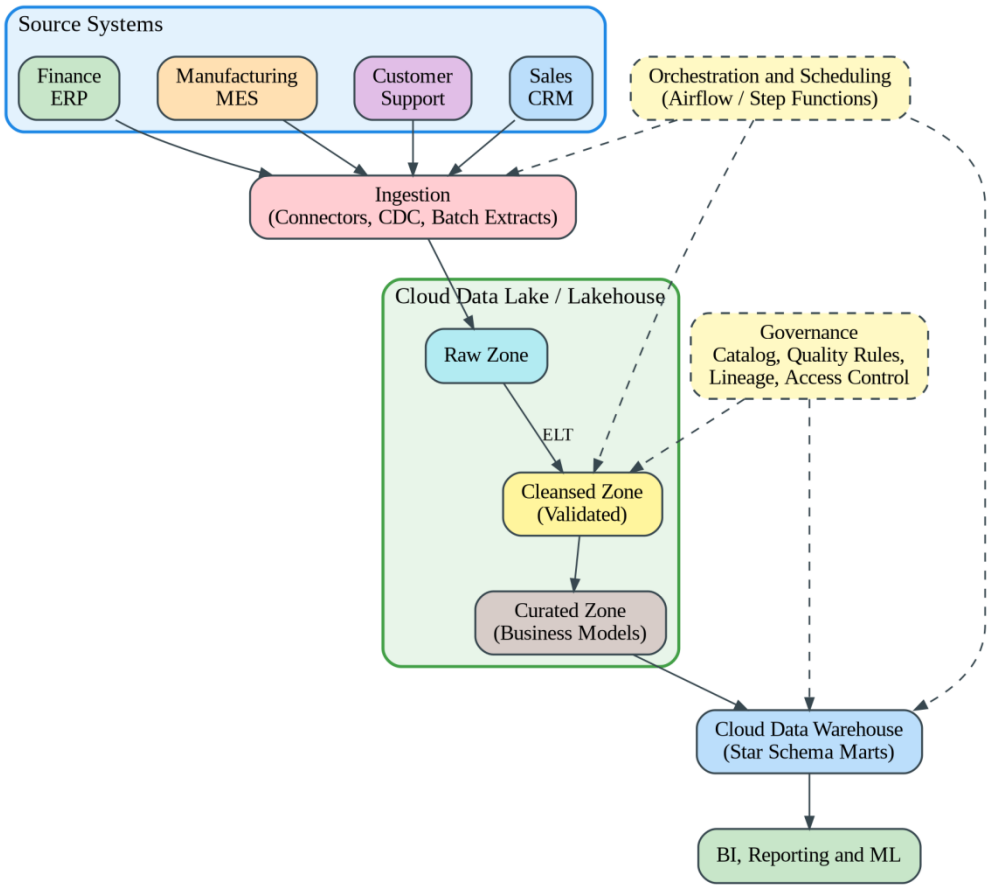


Figure 9: Cloud ELT Pipeline with Orchestration and Governance

2. Data Quality, Metadata Management and Governance

Data quality: Automated rules check completeness, uniqueness, validity, consistency and timeliness at each stage (tools such as Great Expectations, Deequ or dbt tests). Bad records go to a quarantine area, and failures raise alerts. Master data management creates a single consistent view of customers and products, and common definitions and reference data keep metrics consistent across regions.

Metadata management: A data catalog (AWS Glue Data Catalog, Azure Purview, Collibra, Alation) stores technical metadata (schemas), business metadata (definitions, owners) and operational metadata (run times, row counts). Data lineage shows where each field came from and which reports depend on it, which helps impact analysis and audits.

Governance: Named data owners and stewards, classification of sensitive data, role-based and column-level access, masking of personal data, retention rules and compliance with regional laws such as GDPR. Policies are documented and enforced through the catalog and access controls.

3. Workflow Orchestration and Scheduling

Orchestration tools (Apache Airflow, AWS Step Functions, Azure Data Factory, Cloud Composer, Dagster) define each pipeline as a directed acyclic graph of tasks with dependencies. They provide the following:

- Scheduling by time or by event, such as the arrival of a file.
- Dependency management, so transformations start only after extraction and quality checks succeed.
- Retries, timeouts, backfills and alerting on failure.
- Parameterisation across regions and environments.
- Monitoring, logging and run history for audit and SLA tracking.

4. Optimizing Query Performance and Storage Costs

Goal	Techniques
Query performance	Partitioning and clustering on common filters, columnar storage, materialized views and result caching, pre-aggregated marts, appropriate distribution or sort keys, avoiding SELECT *, workload management and right-sized compute.
Storage cost	Compressed columnar formats (Parquet), lifecycle rules to move old data to cheaper tiers, deleting temporary and duplicate data, retention limits, and incremental processing rather than full reloads.
Overall cost	Auto-suspend and auto-scale warehouses, separation of storage and compute, usage monitoring, and chargeback by department.

Question 5: Machine Learning Platform for Delivery Delay Prediction

Scenario: A logistics company wants to predict delivery delays using historical shipment data, weather information, and live GPS tracking. The analytics platform must support continuous model training and real-time predictions.

1. Cloud-Based Machine Learning and Analytics Architecture

- **Data sources:** Historical shipment records, weather APIs and live GPS streams from vehicles.
- **Ingestion:** Streaming through Kafka or Kinesis for GPS and weather feeds, and batch loads for history.
- **Storage:** A data lake holds raw and curated data, and a feature store keeps reusable features for training (offline) and serving (online).
- **Processing:** Spark for large-scale feature computation, and a stream processor for real-time features such as average speed over the last 10 minutes.
- **Training:** Distributed training on managed ML services (SageMaker, Vertex AI, Azure ML) with experiment tracking.
- **Deployment:** A model registry plus real-time endpoints for per-shipment predictions and batch scoring for daily planning.
- **Consumption:** Dispatch dashboards, customer notifications and route re-planning systems.

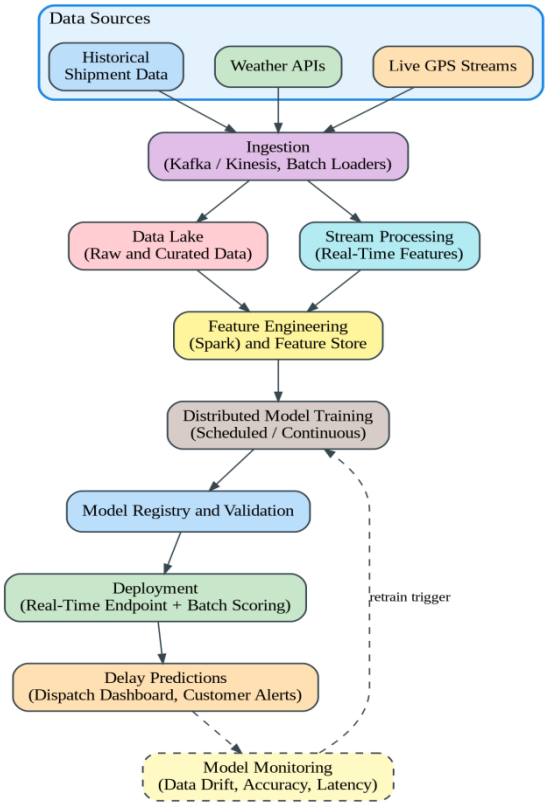


Figure 10: Cloud ML Architecture for Delivery Delay Prediction

2. Big Data Technologies for Training and Inference

Training: Historical data is too large for a single machine, so Spark, Dask or distributed training frameworks (Horovod, PyTorch Distributed, XGBoost on Spark) split data and computation across a cluster, and GPU instances accelerate deep models. Typical models include gradient boosting for tabular features and sequence models for GPS trajectories. Hyperparameter tuning runs many trials in parallel.

Inference: For real time, a stream processor joins incoming GPS events with weather and shipment features from the online feature store and calls the model endpoint, returning a delay probability within milliseconds to seconds. Autoscaling endpoints handle peak load. For non-urgent use cases, batch inference scores all open shipments on a schedule.

3. Cloud-Native AI/ML Services for Deployment and Monitoring

Stage	Cloud-Native Capability
Build	Managed notebooks, AutoML, built-in algorithms, experiment tracking.
Deploy	Model registry with versioning and approval, one-click real-time endpoints, serverless inference, canary and blue-green rollouts, A/B testing.
Pipeline	ML pipelines (SageMaker Pipelines, Kubeflow, Vertex Pipelines) automating data preparation, training, evaluation and deployment (MLOps).
Monitor	Tracking of latency, errors, data drift, concept drift and prediction accuracy against actual delivery outcomes, with alerts and automatic retraining triggers.
Govern	Model explainability, bias checks, audit trails and access control.

4. Strategies for Data Ingestion, Feature Engineering and Model Updates

- **Large-scale ingestion:** Partitioned streaming topics for parallelism, schema registry, back-pressure handling, micro-batching and exactly-once or idempotent processing, with a dead-letter path for bad records.
- **Feature engineering:** Build features once in a feature store so training and serving use identical definitions (avoiding training-serving skew). Examples include distance remaining, traffic and weather severity, time of day, carrier history and rolling averages. Handle missing GPS points through interpolation and aggregate in time windows.
- **Model updates:** Schedule retraining (for example nightly or weekly) and trigger it additionally when drift is detected. Validate each new model against the current one on holdout data, then deploy gradually through canary or shadow mode and keep the ability to roll back quickly.
- **Cost and reliability:** Use Spot instances for training, autoscaling for serving, and keep versioned datasets and models for reproducibility.

SET 3

Question 1: Cloud Migration for a Manufacturing Company

Scenario: A manufacturing company plans to migrate its legacy enterprise applications and databases from an on-premises data center to the cloud. The organization wants to minimize downtime while ensuring application performance and security.

1. Cloud Migration Strategy

A phased, wave-based migration is used instead of a single big-bang move, so risk and downtime stay small.

- **Discover and assess:** Inventory all applications, servers and databases, map dependencies, measure performance baselines and identify compliance needs. Tools such as AWS Application Discovery Service or Azure Migrate help.
- **Plan:** Decide the target for each application (the 6 Rs), define migration waves (low-risk applications first), and set success criteria, rollback plans and a business-friendly cutover window.
- **Prepare the landing zone:** Build the network, identity, security baseline, logging and cost controls in the cloud, and set up a secure link (VPN or dedicated connection).
- **Pilot:** Migrate a non-critical application to validate tooling and processes.
- **Migrate in waves:** Use continuous data replication (such as AWS DMS or Azure Database Migration Service) so that the final cutover requires only minutes of downtime.
- **Cutover, validate and optimise:** Switch traffic, monitor closely, keep the old system as fallback, then decommission it and right-size resources.

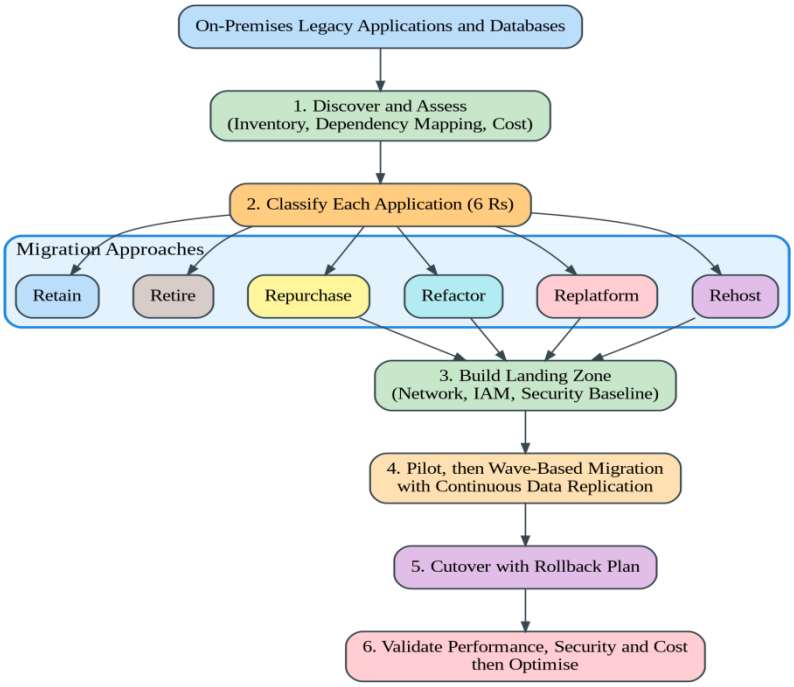


Figure 11: Phased Cloud Migration Strategy Using the 6 Rs

2. Applying the Migration Approaches (6 Rs)

Approach	Meaning	Example for the Manufacturing Company
Rehost (lift and shift)	Move servers as they are to cloud virtual machines.	Stable ERP servers moved quickly with minimal change, as a first step.
Replatform (lift, tinker and shift)	Small optimisations without changing the core design.	Move the on-premises Oracle or SQL Server to a managed database service.
Refactor / re-architect	Redesign for cloud-native features.	Break the production-monitoring application into microservices with auto-scaling.
Repurchase	Replace with a SaaS product.	Replace the old CRM or HR system with a SaaS solution.
Retire	Decommission applications no longer needed.	Switch off duplicate or unused reporting tools to reduce cost and risk.
Retain	Keep on-premises for now.	Shop-floor systems with ultra-low latency or hardware dependency, or compliance constraints.

3. Validating Application Performance After Migration

- Compare against the pre-migration baseline: response time, throughput, error rate, CPU, memory and database query times.
- Run functional, integration and regression tests, and user acceptance testing with business users.
- Run load and stress tests that simulate peak production volumes.
- Use application performance monitoring and synthetic transactions to watch real behaviour.
- Verify data integrity with row counts, checksums and reconciliation reports.
- Check security configuration, access control and backup and recovery operation.
- Review cost against the estimate and tune instance sizes, caching and database settings.

4. Reducing Migration Risks and Downtime

Risk	Mitigation
Extended downtime	Continuous replication, cutover during low-activity windows, blue-green or parallel run, and DNS switch with low time-to-live.
Data loss or corruption	Backups before migration, validation checks, and keeping the source untouched until sign-off.
Performance problems	Right-sizing from baselines, pilot testing, and load testing before cutover.
Security gaps	Landing-zone security baseline, encryption, least-privilege IAM, and security review of each wave.

Risk	Mitigation
Unknown dependencies	Thorough discovery and dependency mapping before moving anything.
Failure of the migration	Documented rollback plan, rehearsals and a go/no-go checklist.
Skills and change resistance	Training, clear communication and use of an experienced migration partner.

Question 2: IoT Analytics Platform for a Smart City

Scenario: A smart city project deploys thousands of IoT sensors to monitor traffic, air quality, energy consumption, and public safety. The collected data must be processed and analyzed in real time for decision-making.

1. Cloud-Based Architecture for IoT Data Collection and Analytics

- **Device layer:** Sensors and cameras with unique identities and certificates, sending data through low-power protocols (MQTT, LoRaWAN, NB-IoT, 5G).
- **Edge layer:** Gateways aggregate data, filter noise, compress it, buffer during network outage and run simple local analytics (for example, a camera detecting an accident), reducing latency and bandwidth.
- **Ingestion layer:** A cloud IoT service (AWS IoT Core, Azure IoT Hub, Google IoT alternatives) with a message broker authenticates devices and routes messages.
- **Processing layer:** Stream processing engines (Kafka Streams, Flink, Spark Streaming, Kinesis Analytics) clean, enrich and analyse data in real time.
- **Storage layer:** A time-series database for recent, fast queries (hot), and a data lake for long-term history (cold).
- **Application layer:** Dashboards, alerts, public APIs and machine learning models for traffic prediction and pollution forecasting.

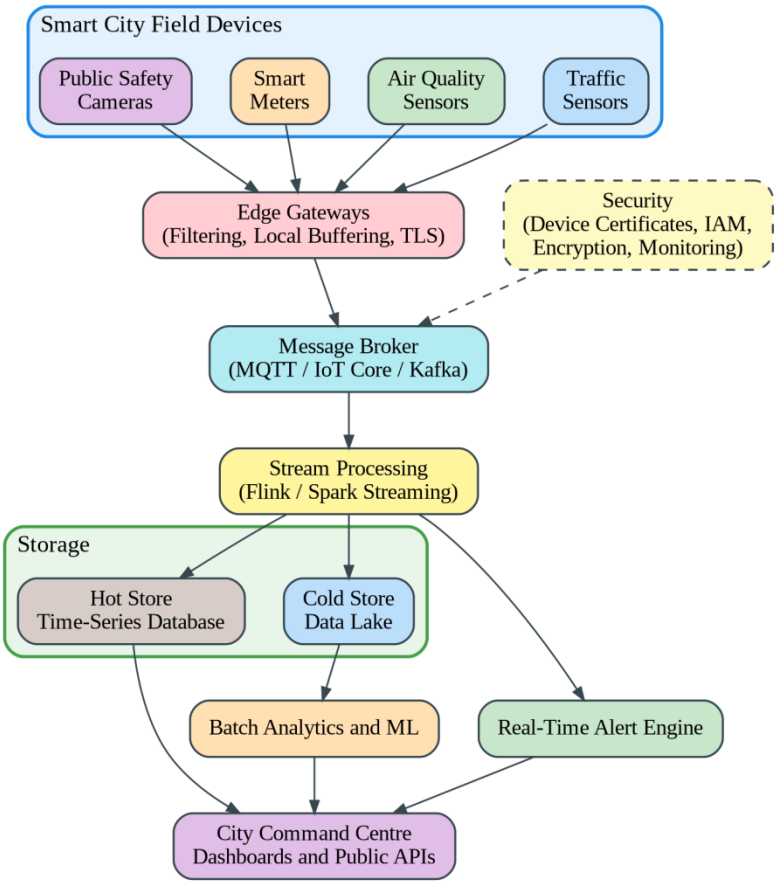


Figure 12: Smart City IoT Analytics Architecture

2. Integration of Message Brokers, Stream Processing and Cloud Storage

Sensors publish readings to topics on the message broker (for example, city/zone5/air/pm25). The broker decouples producers from consumers, buffers bursts and delivers messages to several subscribers. A rules engine can route messages directly: critical readings go to the alert service, all readings go to the stream processor.

The stream processor consumes from the broker and performs windowed aggregations (average traffic per minute), joins with reference data (sensor location), anomaly detection and threshold checks. Results are written to the time-series database for live dashboards, and raw plus processed data is written to the data lake in partitioned Parquet files for batch analytics, machine learning and audits. Alerts are pushed to emergency and city operations teams through notification services.

3. Handling High-Volume, High-Velocity Sensor Data

- Partitioned topics and parallel consumers to spread load, with horizontal scaling of brokers and processors.
- Edge filtering, deduplication and aggregation to send only useful data.
- Efficient binary formats (Avro, Protobuf) and compression.
- Windowing and watermarks to deal with late and out-of-order events.

- Back-pressure and buffering using queues so spikes do not overload downstream systems.
- Tiered storage and downsampling: keep raw data for a short time and aggregates for years.
- Time-series optimised databases and indexes for quick queries.

4. Scalability, Reliability and Security Techniques

Area	Techniques
Scalability	Managed autoscaling services, serverless processing, partitioning and stateless consumers, edge computing to offload the cloud.
Reliability	Multi-AZ deployment, replicated brokers and databases, message persistence and retries, dead-letter queues, edge buffering, health monitoring and disaster recovery.
Security	Unique device identity with X.509 certificates, mutual TLS, secure boot and signed firmware updates, device registry and revocation, least-privilege policies, network segmentation, encryption at rest, anomaly detection and privacy protection (anonymising camera data) in line with regulations.

Question 3: Business Intelligence for a Global E-Commerce Company

Scenario: A global e-commerce company wants executives to monitor sales performance, customer behavior, and regional trends through interactive dashboards. The solution must support real-time analytics and historical reporting.

1. Cloud-Based Business Intelligence Architecture

- **Data sources:** Order and payment systems, CRM, web and app clickstream, inventory and regional ERP systems.
- **Ingestion:** A streaming pipeline (Kafka/Kinesis) provides near-real-time sales and behaviour events, and batch ELT jobs load full historical data on a schedule.
- **Storage and modelling:** A cloud data warehouse (BigQuery, Redshift, Snowflake, Synapse) with a star schema: fact tables for sales and events and dimensions for product, customer, region and time. Aggregate tables and materialized views support fast summary queries.
- **Semantic layer:** Common business definitions (revenue, conversion, churn) so every dashboard shows the same numbers.
- **Presentation:** Power BI, Tableau, Looker or QuickSight dashboards with drill-down, filters, maps and mobile access for executives.
- **Real-time plus historical:** Recent data is updated continuously (streaming inserts), while history is stored in the warehouse and combined in the same dashboard.

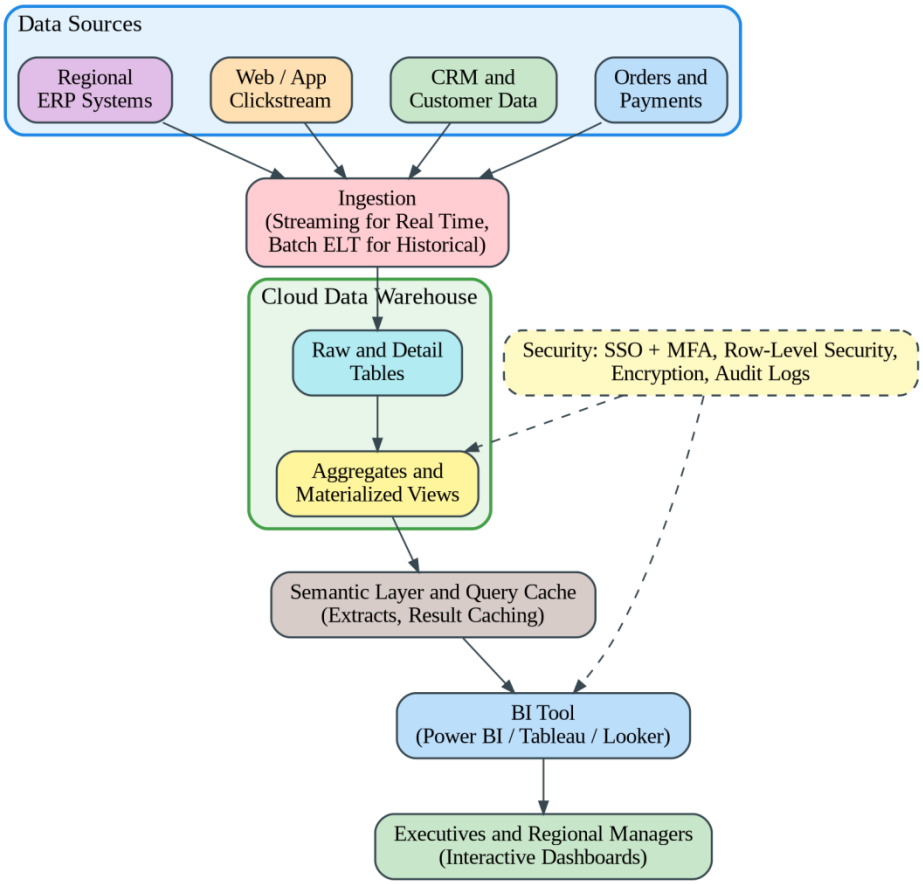


Figure 13: Cloud Business Intelligence Architecture for Executive Dashboards

2. Integrating Visualization Tools with Cloud Data Warehouses

BI tools connect using native connectors or ODBC/JDBC drivers with secure authentication (SSO, OAuth or service accounts). There are two main modes:

Mode	How It Works	Best For
Live / direct query	The dashboard sends SQL to the warehouse each time and shows current data.	Real-time monitoring and very large datasets.
Import / extract	Data is cached in the BI tool's in-memory engine and refreshed on a schedule.	Fast interactive dashboards with data that is not changing every second.

Many deployments use a hybrid, with live queries for operational tiles and extracts for historical analysis. Row-level security rules and shared semantic models are passed through so that the warehouse enforces permissions.

3. Optimizing Dashboard Performance with Large Datasets

- Pre-aggregate data into summary tables or materialized views at the grain needed (daily by region) instead of querying billions of raw rows.
- Partition and cluster warehouse tables by date and region, and use columnar storage.
- Enable query result caching and BI extracts, and refresh them incrementally.

- Limit the number of visuals per dashboard, apply default filters (such as last 30 days) and use drill-through for details.
- Avoid heavy calculations in the visualization layer, and push them down to SQL.
- Scale warehouse compute for peak hours and use workload isolation so dashboards do not compete with ETL jobs.
- Monitor slow queries and improve them regularly.

4. Security Measures to Protect Reports and Sensitive Data

Measure	Description
Authentication	Single sign-on with multi-factor authentication.
Authorization	Role-based access, row-level security (a regional manager sees only their region) and column-level security or masking for personal data.
Encryption	TLS for connections and encryption at rest in the warehouse and BI storage, with managed keys.
Network	Private connectivity, IP allow-lists and VPN for BI services.
Governance and audit	Data classification, usage and access logging, certified datasets, and controlled sharing and export.
Compliance	Data residency and privacy rules such as GDPR, including anonymising customer data in reports.

Question 4: Monitoring and Performance Optimization for an Online Gaming Company

Scenario: An online gaming company experiences fluctuating workloads throughout the day. During peak hours, users report high latency and reduced application performance.

1. Cloud Monitoring and Performance Optimization Solution

The solution is built around an observability platform and an automated feedback loop of measuring, alerting, acting and learning.

- **Instrumentation:** Agents and SDKs on game servers, load balancers, databases and caches, with client-side telemetry measuring real player latency (ping), frame rate and login time.
- **Observability platform:** Amazon CloudWatch, Azure Monitor, Google Cloud Operations, or Prometheus with Grafana, Datadog or similar, plus distributed tracing (X-Ray, Jaeger, OpenTelemetry).
- **Dashboards and alerts:** Live views by region and game server, alerts on service-level objectives (such as 95th percentile latency below 100 ms).
- **Automated response:** Auto-scaling policies, automatic server restarts and traffic shifting.

- **Optimisation of the delivery path:** Servers in regions close to players, a CDN for game assets and patches, in-memory caching for leaderboards and sessions, and database read replicas.

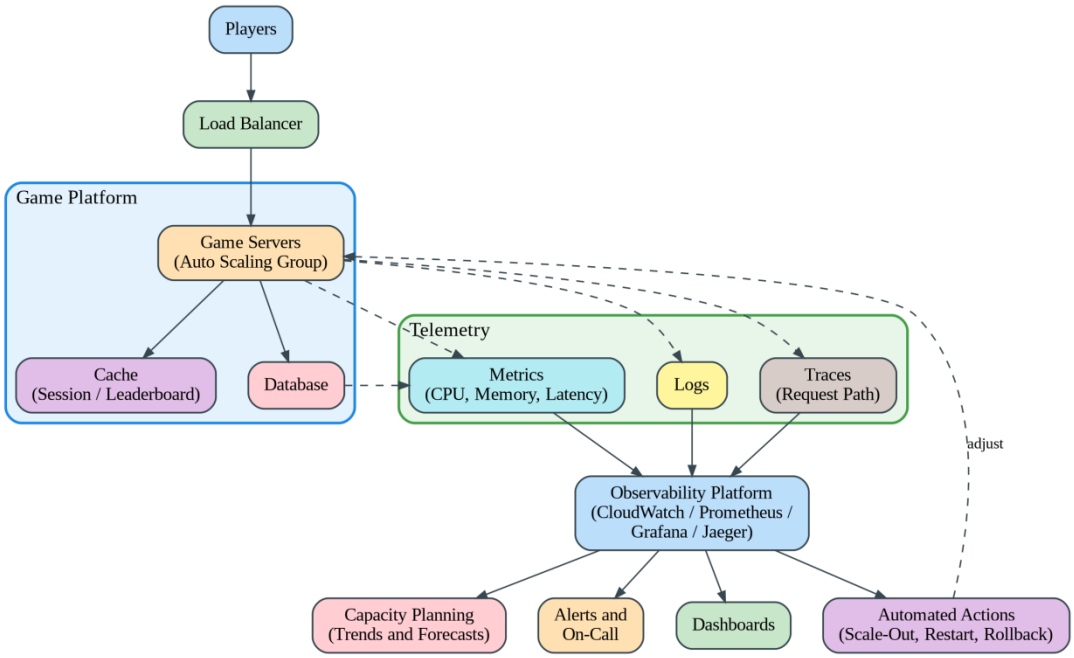


Figure 14: Cloud Monitoring and Performance Optimization Feedback Loop for an Online Game

2. How Monitoring Tools, Metrics, Logs and Tracing Identify Bottlenecks

Signal	What It Shows	Example in This Scenario
Metrics	Numeric time series: CPU, memory, network, request rate, latency percentiles, connection counts.	CPU on game servers reaches 90 percent every evening at 8 pm, showing a compute bottleneck.
Logs	Detailed event records and errors.	Logs show database connection timeouts and match-making errors during peak.
Traces	The path and time of a single request through all services.	A trace shows that most of the 800 ms login time is spent in the profile database call.

By correlating the three signals on the same timeline, engineers move from symptom (high latency) to cause (an under-sized database or insufficient servers) quickly. Comparing metrics before and after a release, and breaking them down by region, server and game mode, isolates the problem.

3. Resource Optimization and Capacity Planning

- **Right-sizing:** Match instance types to workload (compute-optimised or GPU where needed) using utilisation data.
- **Auto-scaling:** Target tracking on CPU or concurrent players, scheduled scaling for the daily peak and for events such as new season launches, and predictive scaling from history.

- **Efficiency:** Tune code and queries, add caching, use connection pooling, and compress network traffic.
- **Capacity planning:** Analyse growth trends and seasonality, run load tests at 1.5 to 2 times expected peak, and keep headroom (for example 20 to 30 percent) and quotas raised in advance.
- **Pre-warming** capacity before known events instead of waiting for scale-out.

4. Improving Availability While Controlling Operational Costs

Availability Method	Cost Control
Multi-AZ and multi-region deployment with load balancing and health checks.	Reserved or Savings Plan capacity for the baseline, on-demand or Spot for the peak and non-critical services.
Automatic failover for databases, and redundant caches.	Scale in aggressively during off-peak hours, and shut down idle test environments.
Graceful degradation and queueing (for example, disabling non-essential features at extreme load).	Use a CDN to cut bandwidth and origin load, and compress data.
Rolling and canary deployments to avoid downtime during updates.	Tag resources, set budgets and alerts, and review cost reports regularly.

Question 5: Data Lifecycle Management for a Financial Institution

Scenario: A financial institution stores petabytes of customer transaction data for regulatory compliance and business analytics. The organization must retain historical records while managing storage costs efficiently.

1. Cloud-Based Data Lifecycle Management Strategy

Data lifecycle management governs data from creation through active use, archival and final deletion, with each stage placed on the most economical storage that still meets access, security and regulatory needs.

Stage	Typical Age	Storage Tier	Characteristics
Hot	0 to 90 days	Standard storage / SSD, warehouse tables	Frequent access, analytics and customer queries, highest cost.
Warm	3 to 12 months	Infrequent-access tier	Occasional access, lower cost, millisecond retrieval.
Cold	1 to 3 years	Archive tier (Glacier, Azure Archive, Coldline)	Rare access, minutes to hours to retrieve, low cost.
Deep archive	3 to 7+ years	Deep archive with WORM lock	Compliance retention only, lowest cost, retrieval in hours.
Disposal	After retention ends	Secure deletion	Cryptographic erasure and certificate of destruction, unless a legal hold applies.

Lifecycle rules automatically transition objects between tiers and delete them at the end of retention. Intelligent tiering can also move data automatically based on access patterns.

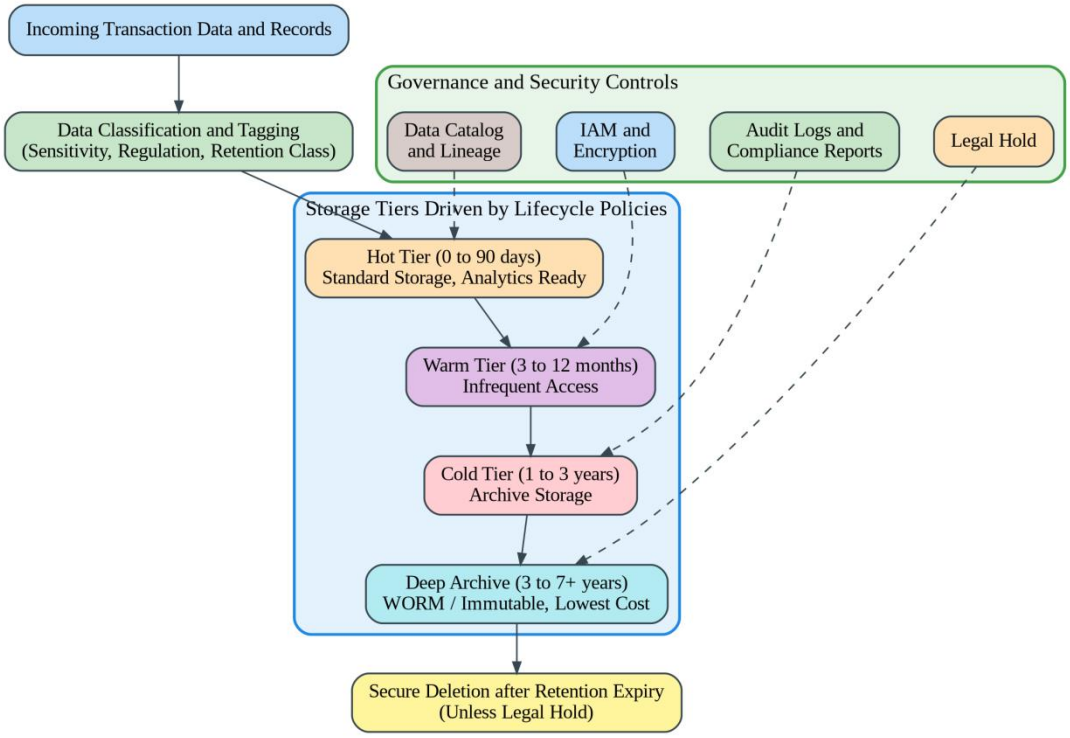


Figure 15: Data Lifecycle Management with Tiered Storage and Governance Controls

2. Data Classification, Retention Policies and Archival Storage

- **Data classification:** Label data by sensitivity (public, internal, confidential, restricted) and by type (transactions, account data, personal identifiers, logs) using automated discovery tools and metadata tags. Classification decides encryption, access control, location and retention.
- **Retention policies:** Map each data class to legal and business retention requirements (for example, several years for transaction records under financial regulations), documented by compliance and legal teams and applied as automated lifecycle rules. Legal holds suspend deletion during investigations.
- **Archival storage:** Use immutable, write-once-read-many object lock for records that must not be altered, store data in compact columnar formats with compression, keep a catalog or index so archived data can be found and restored, and replicate archives to a second region for durability.
- **Restoration and testing:** Define retrieval service levels and test restores periodically.

3. Importance of Governance, Auditing and Regulatory Compliance

In financial services, data is both an asset and a liability. Governance defines ownership, quality standards, access policies and accountability. Auditing proves that policies are followed. Compliance avoids fines, legal action and loss of customer trust.

- **Regulations:** Rules such as PCI DSS, SOX, GDPR, Basel requirements and local regulator guidelines require retention, confidentiality, integrity and the right to erasure where applicable.
- **Auditing:** Immutable logs of who accessed, changed or deleted data, retained and reviewed regularly, and available to auditors.
- **Governance practices:** Data catalog and lineage, data owners and stewards, data quality rules, periodic access reviews and policy as code.
- **Big data challenge:** At petabyte scale, manual control is impossible, so classification, retention and monitoring must be automated.

4. Balancing Accessibility, Security and Cost Optimization

Objective	**Techniques**
Accessibility	Keep recent data on fast tiers, maintain a searchable catalog, use query-in-place engines (Athena, Spectrum, BigQuery external tables) so analysts can query archives without full restore, and cache or restore frequently requested archives.
Security	Encryption at rest and in transit with managed keys, least-privilege IAM, MFA, immutable storage, separate accounts for archives, masking and tokenisation, and continuous monitoring.
Cost	Automated tiering, compression and columnar formats, deletion of redundant and expired data, avoiding unnecessary replication, reserved capacity, and cost dashboards with tagging.
Balance	Review access patterns regularly and adjust lifecycle thresholds, because moving data too early causes costly retrievals while moving too late wastes storage spend.